PYTHON IN DATABRICKS CHEAT SHEET

Production Reference: DBUtils, Delta Live Tables (DLT), pandas on Spark, SDK & Magic Commands

PYTHON 3.10+ | DBUTILS | DELTA LIVE TABLES | DATABRICKS SDK

Databricks Utilities (dbutils) Master Reference

dbutils

```python
# 1. File System (dbutils.fs) & Volumes
dbutils.fs.ls("/Volumes/prod/finance/raw_files/")
dbutils.fs.cp("abfss://landing@adls.dfs.core.windows.net/data.csv",
              "/Volumes/prod/finance/raw_files/", recurse=True)
dbutils.fs.rm("/tmp/scratch/", recurse=True)
dbutils.fs.put("/tmp/config.json", '{"env":"prod"}', overwrite=True)

# 2. Secrets & Azure Key Vault
secret_token = dbutils.secrets.get(scope="akv-scope", key="storage-key")
scopes = dbutils.secrets.listScopes()

# 3. Notebook Workflows
res = dbutils.notebook.run("sub_pipeline", timeout_seconds=1800,
                           arguments={"batch_id": "101"})
dbutils.notebook.exit("SUCCESS: 154000 records processed")

# 4. Interactive Widgets
dbutils.widgets.text("env", "dev", "Target Environment")
env_val = dbutils.widgets.get("env")
```

Delta Live Tables (DLT) Declarative Pipelines

DLT

```python
import dlt
from pyspark.sql import functions as F

# Streaming Ingestion Table
@dlt.table(
    name="bronze_txns",
    comment="Raw transaction stream",
    table_properties={"quality": "bronze"}
)
def bronze_txns():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "json")
            .load("abfss://landing@adls.dfs.core.windows.net/txns/"))

# Silver Table with Data Quality Expectations
@dlt.table(comment="Validated transactions")
@dlt.expect_or_drop("valid_id", "txn_id IS NOT NULL")
@dlt.expect_or_fail("valid_amt", "amount > 0.0")
def silver_txns():
    return dlt.read_stream("bronze_txns").filter(F.col("status") != "REJECT")
```

DLT Expectations: `expect` (logs metric), `expect_or_drop` (drops bad rows), `expect_or_fail` (fails pipeline on violation).

CDC Ingestion with dlt.apply_changes()

SCD Type 1 & 2

```python
import dlt

# Automatic SCD Type 2 History Tracking
dlt.create_streaming_table("gold_customers_scd2")

dlt.apply_changes(
    target="gold_customers_scd2",
    source="silver_customer_cdc",
    keys=["customer_id"],
    sequence_by="updated_timestamp",
    apply_as_deletes="operation = 'DELETE'",
    except_column_list=["operation"],
    stored_as_scd_type=2 # Auto creates __START_AT and __END_AT
)
```

- `stored_as_scd_type=1` overwrites records in place (latest snapshot).
- `stored_as_scd_type=2` manages effective date ranges and active flags out-of-the-box!

pandas API on Spark (pyspark.pandas)

Scale pandas

```python
import pyspark.pandas as ps

# Read distributed data using familiar pandas syntax
psdf = ps.read_parquet("abfss://gold@adls.../sales/")

# Distributed GroupBy & Summary Aggregations
summary = psdf.groupby("country")["revenue"].agg(["mean", "sum", "count"])

# Seamless Two-Way Conversion
spark_df = psdf.to_spark()
ps_df_again = spark_df.to_pandas_on_spark()
```

Anti-Pattern: Avoid calling `.to_pandas()` or `.iloc` on large DataFrames as it pulls all distributed data into driver RAM, risking OOM!

Databricks Connect v2 & Databricks Python SDK

Developer Tooling

```python
# Databricks Connect v2 (Local IDE to Remote Cluster over gRPC)
from databricks.connect import DatabricksSession
spark = DatabricksSession.builder.getOrCreate()
df = spark.read.table("prod_catalog.silver.transactions")
df.filter("amount > 1000").show()

# Databricks Python SDK (Workspace Automation)
from databricks.sdk import WorkspaceClient
w = WorkspaceClient()
run = w.jobs.run_now(job_id=892145)
clusters = w.clusters.list()
```

Thin Client: Databricks Connect v2 sends unresolved DataFrame plans via gRPC; execution runs entirely on the remote Databricks cluster.

Databricks Magic Commands & Scoping

Notebook Magic

Magic	Language / Action	Behavior & Scope
`%python`	Python 3	Default language; shares local variables across notebook cells
`%sql`	Spark SQL	Executes SQL query; accessible via `_sqldf` DataFrame
`%sh`	Bash Shell	Runs bash on the driver node container
`%fs`	Databricks FS	Shorthand for `dbutils.fs` (e.g. `%fs ls /Volumes`)
`%pip`	pip install	Installs library scoped to the current notebook session only
`%run`	Sub-notebook	Executes another notebook in-line and inherits all its variables

%run vs dbutils.notebook.run: `%run` shares memory & variables in the same session; `dbutils.notebook.run` executes in an isolated ephemeral job context.

🐍 PYTHON IN DATABRICKS CHEAT SHEET

Wheels, Databricks Asset Bundles (DABs), Observability, Resilience, Types & Unit Testing

DABS / CI/CD | PRODUCTION WHEELS | OBSERVABILITY

📦 Wheels & Unity Catalog Volumes

Packaging

```
# In-Notebook ephemeral dependency install
%pip install pydantic==2.8.2 tenacity==8.4.1

# Production Wheel Distribution via Unity Catalog Volumes
# Store built wheel in governed volume:
# /Volumes/prod_catalog/core_libs/my_etl-1.2.0-py3-none-any.whl
%pip install /Volumes/prod_catalog/core_libs/my_etl-1.2.0-py3-none-any.whl

# Import modular reusable package in notebook
from my_etl.transformers import BankingNormalizer
normalizer = BankingNormalizer(currency="EUR")
```

Volumes for Libraries: Replacing legacy DBFS root with Unity Catalog Volumes ensures strict access control and auditing for third-party binary artifacts.

🚀 Databricks Asset Bundles (DABs) CI/CD

DevOps

```
# databricks.yml definition snippet
bundle:
  name: bfsi_settlement_pipeline

targets:
  dev:
    workspace:
      host: https://adb-dev.azuredatabricks.net
  prod:
    workspace:
      host: https://adb-prod.azuredatabricks.net

resources:
  jobs:
    nightly_settlement:
      name: "[prod] Nightly Settlement Job"
      tasks:
        - task_key: run_dlt
          pipeline_task:
            pipeline_id: 489a-bc12-dlt-id
```

CLI Command	Action
`databricks bundle validate`	Lints and verifies bundle syntax
`databricks bundle deploy -t prod`	Deploys resources to target workspace
`databricks bundle run nightly_settlement`	Executes workflow immediately

📊 Structured Logging & Observability

Observability

```
import json, logging
from datetime import datetime

class StructuredPipelineLogger:
    def __init__(self, pipeline_name: str):
        self.name = pipeline_name

    def log_event(self, event_name: str, records: int, duration_sec: float):
        payload = {
            "timestamp": datetime.utcnow().isoformat(),
            "pipeline": self.name,
            "event": event_name,
            "records_processed": records,
            "duration_sec": duration_sec
        }
        print(json.dumps(payload)) # Ingested into Azure Log Analytics

audit = StructuredPipelineLogger("SEPA_Payment_Processor")
audit.log_event("STAGE_COMPLETED", records=845000, duration_sec=14.2)
```

🔄 Concurrent Extraction & Exponential Retries

Resilience

```
from concurrent.futures import ThreadPoolExecutor
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=2, max=10))
def fetch_api_page(page_id: int) -> dict:
    # Resilient REST API caller
    return {"page": page_id, "data": [1, 2, 3]}

# Parallelize lightweight API calls on Driver before Spark processing
with ThreadPoolExecutor(max_workers=8) as executor:
    results = list(executor.map(fetch_api_page, range(1, 50)))
```

Thread Concurrency: Use Python `ThreadPoolExecutor` on the driver for I/O-bound REST calls, then parallelize the dataset with `spark.createDataFrame()`.

🛡️ Type Annotations & Schema Contracts

Data Quality

```
from pyspark.sql import DataFrame, Column
from typing import Callable

def apply_curation(
    df: DataFrame,
    primary_key: str,
    business_rule: Callable[[Column], Column]
) -> DataFrame:
    assert primary_key in df.columns, f"Missing required key: {primary_key}"
    return df.filter(business_rule(df[primary_key]))
```

- Enforce PEP 484 type hints across modular libraries.
- Pre-flight assertions catch schema mismatch errors in milliseconds before distributed stages start.

🧪 Unit Testing PySpark Pipelines in CI/CD

Testing

```
import pytest
from pyspark.testing.utils import assertDataFrameEqual

def test_filter_active(spark):
    input_data = [(1, "ACTIVE"), (2, "CANCELLED")]
    input_df = spark.createDataFrame(input_data, ["id", "status"])

    expected_data = [(1, "ACTIVE")]
    expected_df = spark.createDataFrame(expected_data, ["id", "status"])

    result_df = input_df.filter("status = 'ACTIVE'")
    # Spark 3.5+ built-in test assertion
    assertDataFrameEqual(result_df, expected_df)
```

pyspark.testing: Use `assertDataFrameEqual` and `assertSchemaEqual` in pytest to validate transformations without third-party libraries.